

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №1
« Експериментальна оцінка ентропії на символ
джерела відкритого тексту »

Виконали: Ромашко Мирослава та
Чупай Яна, група ФБ-41
Перевірила: Селюх Поліна Валентинівна

Мета: Засвоєння понять ентропії на символ джерела та його надлишковості, вивчення та порівняння різних моделей джерела відкритого тексту для наближеного визначення ентропії, набуття практичних навичок щодо оцінки ентропії на символ джерела.

ХІД РОБОТИ

1. Оберіть довільний текст російською мовою обсягом не менше 1 МБ. Виконайте попередню обробку тексту відповідно до таких правил:
 - усі літери привести до нижнього регістру;
 - символи, що не належать до заданого алфавіту, вилучити або замінити пробілами;
 - послідовності пробілів та інших розділових символів трактувати як один пробіл;
 - для обчислень без пробілів вилучити всі пробіли з попередньо нормалізованого тексту.

Зафіксуйте правила попередньої обробки, використані у вашій програмі.

Ми обрали текст **“Пять ложек эликсира”**

Size: 1.27 MB (1,339,401 bytes)

```
1 import re
2
3 RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцчшщъыюя"
4 ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")
5
6 def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
7     text = raw_text.lower()
8     text = text.replace("ё", "е")
9     text = text.replace("ь", "ъ")
10    text = text.replace("э", "е")
11    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
12    text = re.sub(r" +", " ", text).strip()
13
14    if not keep_spaces:
15        text = text.replace(" ", "")
16    return text
17
18 filename = "Pyat_lojek_eliksira.txt"
19
20 try:
21     with open(filename, encoding="utf-8", errors="ignore") as file:
22         raw = file.read()
23
24     text_with_spaces = preprocess(raw, keep_spaces=True)
25     text_no_spaces = preprocess(raw, keep_spaces=False)
26
27     print(f"Довжина сирого тексту: {len(raw)}")
28     print(f"Довжина після нормалізації (з пробілами): {len(text_with_spaces)}")
29     print(f"Довжина без пробілів: {len(text_no_spaces)}")
30     print(f"\nПерші 300 символів після обробки (з пробілами):")
31     print(text_with_spaces[:300])
32 except FileNotFoundError:
33     print(f"Помилка: Файл '{filename}' не знайдено. Переконайтеся, що він лежить в тій самій папці, що й скрипт.")
```

Наш заданий алфавіт складається з **30 літер**. А всі літери, що до нього не ввійшли (включно з «ё», «ъ» та «э»), завдяки командам `replace` були замінені на відповідні еквіваленти («е», «ь», «е»)

```
Довжина сирого тексту: 742079
Довжина після нормалізації (з пробілами): 690626
Довжина без пробілів: 579587

Перші 300 символів після обробки (з пробілами):
етот том сочинений братьев стругацких составляют пьесы и киносценарии пять ложек еликсира адаптация
льпиниста а сталкер и чародеи популярнейшие киноленты не нуждающиеся в дополнительном представлении
```

Правила обробки:

- нижній регістр;
- заміни літер: «ё»→«е», «ъ»→«ь», «э»→«е» (заданий алфавіт складає 30 літер);
- всі символи поза алфавітом замінені на пробіл;
- повторні пробіли згорнуті в один;
- окремо створено версію без пробілів.

2. Розрахунок статистичних характеристик тексту

Реалізуйте програму для:

1. підрахунку частот окремих символів;
2. підрахунку частот біграм;
3. обчислення N_1 за частотами окремих символів;
4. обчислення N_2 за частотами біграм.

Частоти біграм та відповідне значення N_2 визначте двома способами:

- для біграм, що перетинаються, тобто при послідовному зміщенні на один символ;
- для біграм, що не перетинаються, тобто при зміщенні на два символи.

Обрахунок частот символів і N_1

`clean_text(raw_data, keep_spaces)`: Відповідає за повну нормалізацію тексту згідно з правилами. Вона переводить літери в нижній регістр, виконує словникові заміни («ё»→«е», «ъ»→«ь», «э»→«е»), перетворює всі сторонні символи на пробіли та згортає їхні послідовності.

compute_entropy_h1(freq_dict, total_chars): Рахує ентропію на символ (H_1) за класичною формулою: $H_1 = - \sum p \cdot \log_2(p)$. Функція приймає підраховані частоти символів, ділить кожен частоту на загальну довжину тексту (щоб знайти ймовірність p), а потім накопичує загальну суму ентропії у бітах.

```

1  import math
2  import re
3  from collections import Counter
4
5  CUSTOM_ALPHABET = "абвгдежзийклмнопрстуфхцщщыьюя"
6  ALLOWED_CHARS = set(CUSTOM_ALPHABET + " ")
7
8  def clean_text(raw_data: str, keep_spaces: bool = True) -> str:
9      """Функція для нормалізації тексту за вказаними правилами."""
10     text = raw_data.lower()
11     replacements = {"ё": "є", "ь": "ъ", "э": "е"}
12     for old_char, new_char in replacements.items():
13         text = text.replace(old_char, new_char)
14
15     filtered_chars = [ch if ch in ALLOWED_CHARS else " " for ch in text]
16     text = "".join(filtered_chars)
17     text = re.sub(r"\s+", " ", text).strip()
18
19     if not keep_spaces:
20         text = text.replace(" ", "")
21     return text
22
23  def compute_entropy_h1(freq_dict: Counter, total_chars: int) -> float:
24      """Обчислення ентропії H1 на один символ."""
25     entropy = 0.0
26     for count in freq_dict.values():
27         probability = count / total_chars
28         entropy -= probability * math.log2(probability)
29     return entropy
30
31  filename = "Pyat_lojek_eliksira.txt"
32

```

```

33  try:
34      with open(filename, "r", encoding="utf-8", errors="ignore") as f:
35          original_text = f.read()
36
37      text_spaces = clean_text(original_text, keep_spaces=True)
38      text_nospaces = clean_text(original_text, keep_spaces=False)
39      variants = [
40          ("Текст ІЗ пробілами", text_spaces),
41          ("Текст БЕЗ пробілів", text_nospaces)
42      ]
43
44      for title, processed_text in variants:
45          total_len = len(processed_text)
46          char_counts = Counter(processed_text)
47          h1_value = compute_entropy_h1(char_counts, total_len)
48
49          print(f"\n=== {title} ===")
50          print(f"Загальна довжина: {total_len} символів")
51          print(f"Розмір алфавіту (унікальних символів): {len(char_counts)}")
52          print(f"Ентропія H1: {h1_value:.5f} біт/символ")
53
54          print("Топ-5 найчастіших символів:")
55          for char, count in char_counts.most_common(5):
56              display_char = "[пробіл]" if char == " " else char
57              prob = count / total_len
58              print(f"  '{display_char}': кількість = {count}, ймовірність = {prob:.5f}")
59
60  except FileNotFoundError:
61      print(f"Помилка: Файл '{filename}' не знайдено.")

```

```

=== Текст ІЗ пробілами ===
Загальна довжина: 690626 символів
Розмір алфавіту (унікальних символів): 31
Ентропія H1: 4.35832 біт/символ
Топ-5 найчастіших символів:
'[пробіл]': кількість = 111039, ймовірність = 0.16078
'o': кількість = 62504, ймовірність = 0.09050
'e': кількість = 52549, ймовірність = 0.07609
'a': кількість = 47432, ймовірність = 0.06868
't': кількість = 39121, ймовірність = 0.05665

=== Текст БЕЗ пробілів ===
Загальна довжина: 579587 символів
Розмір алфавіту (унікальних символів): 30
Ентропія H1: 4.43524 біт/символ
Топ-5 найчастіших символів:
'o': кількість = 62504, ймовірність = 0.10784
'e': кількість = 52549, ймовірність = 0.09067
'a': кількість = 47432, ймовірність = 0.08184
't': кількість = 39121, ймовірність = 0.06750
'n': кількість = 38322, ймовірність = 0.06612

```

Далі біграми і H_2 :

```

1  import math
2  import re
3  from collections import Counter
4
5  CUSTOM_ALPHABET = "абвгдежзийклмнопрстуфхцчщьюя"
6  ALLOWED_CHARS = set(CUSTOM_ALPHABET + " ")
7
8  def clean_text(raw_data: str, keep_spaces: bool = True) -> str:
9      """Нормалізація тексту за встановленими правилами."""
10     text = raw_data.lower()
11
12     replacements = {"ё": "е", "ь": "ъ", "э": "е"}
13     for old_char, new_char in replacements.items():
14         text = text.replace(old_char, new_char)
15
16     filtered_chars = [ch if ch in ALLOWED_CHARS else " " for ch in text]
17     text = "".join(filtered_chars)
18     text = re.sub(r"\s+", " ", text).strip()

```

```

19
20     if not keep_spaces:
21         text = text.replace(" ", "")
22
23     return text
24
25 def compute_entropy(counts: Counter, total: int) -> float:
26     """Загальна функція для обчислення ентропії."""
27     entropy = 0.0
28     for count in counts.values():
29         probability = count / total
30         entropy -= probability * math.log2(probability)
31     return entropy
32
33 def get_overlapping_bigrams(text: str) -> list:
34     """Генерація біграм, що перетинаються (зміщення на 1)."""
35     return [text[i:i+2] for i in range(len(text) - 1)]
36
37 def get_nonoverlapping_bigrams(text: str) -> list:
38     """Генерація біграм, що не перетинаються (зміщення на 2)."""
39     return [text[i:i+2] for i in range(0, len(text) - 1, 2)]
40
41 def calculate_h2_per_symbol(bigrams_list: list) -> float:
42     """Обчислення H2 на символ джерела."""
43     counts = Counter(bigrams_list)
44     total_bigrams = sum(counts.values())
45     h_joint = compute_entropy(counts, total_bigrams)
46     return h_joint / 2.0
47
48 filename = "Pyat_lojek_eliksira.txt"
49
50 try:
51     with open(filename, "r", encoding="utf-8", errors="ignore") as f:
52         original_text = f.read()
53         text_spaces = clean_text(original_text, keep_spaces=True)
54         text_nospaces = clean_text(original_text, keep_spaces=False)
55         variants = [
56             ("Текст ІЗ пробілами", text_spaces),
57             ("Текст БЕЗ пробілів", text_nospaces)
58         ]
59
60     for title, processed_text in variants:
61         bg_overlap = get_overlapping_bigrams(processed_text)
62         bg_nonoverlap = get_nonoverlapping_bigrams(processed_text)
63         h2_overlap = calculate_h2_per_symbol(bg_overlap)
64         h2_nonoverlap = calculate_h2_per_symbol(bg_nonoverlap)
65         print(f"\n=== {title} ===")
66         print(f"Кількість перекривних біграм: {len(bg_overlap)}")
67         print(f"Кількість неперекривних біграм: {len(bg_nonoverlap)}")

```

```

68     print(f"H2 (перекривні): {h2_overlap:.5f} біт/символ")
69     print(f"H2 (неперекривні): {h2_nonoverlap:.5f} біт/символ")
70     print("Топ-5 біграм (перекривні):")
71     top_5_overlap = Counter(bg_overlap).most_common(5)
72     for bg, count in top_5_overlap:
73         display_bg = bg.replace(" ", "[ ]")
74         print(f"    '{display_bg}': {count}")
75
76
77 except FileNotFoundError:
78     print(f"Помилка: Файл '{filename}' не знайдено.")

```

```

=== Текст ІЗ пробілами ===
Кількість перекривних біграм: 690625
Кількість неперекривних біграм: 345313
H2 (перекривні): 3.98909 біт/символ
H2 (неперекривні): 3.98853 біт/символ
Топ-5 біграм (перекривні):
'o[ ]': 13485
'e[ ]': 12126
'[ ]п': 11525
'[ ]н': 11215
'[ ]в': 10957

```

```

=== Текст БЕЗ пробілів ===
Кількість перекривних біграм: 579586
Кількість неперекривних біграм: 289793
H2 (перекривні): 4.14824 біт/символ
H2 (неперекривні): 4.14824 біт/символ
Топ-5 біграм (перекривні):
'то': 9277
'ет': 8665
'но': 7105
'ст': 6799
'на': 6758

```

Вираховуємо два варіанти: перекривні біграми та неперекривні

Варіант тексту H2	H2 перекривні	H2 неперекривні	Різниця
З пробілами	3.9891	3.9885	0.00056
Без пробілів	4.1482	4.1482	≈ 0 (0.00000)

Метод обчислення ентропії H_2 за біграмами, що перетинаються (із кроком зміщення на один символ), використовує майже вдвічі більше статистичних даних, оскільки для тексту довжиною N символів він формує вибірку з $N - 1$ біграм. Натомість метод неперекривних біграм (із кроком на два символи) розбиває текст на незалежні пари, генеруючи лише $N/2$ біграм. Незважаючи на цю різницю в обсязі вибірки, розраховані значення H_2 для обох методів виявилися практично ідентичними: для тексту з пробілами розбіжність становить лише 0.00056 біт/символ, а для тексту без пробілів результати збіглися абсолютно (різниця 0.00000). Відсутність суттєвої відмінності пояснюється тим, що досліджений природний текст є достатньо великим і статистично однорідним (стаціонарним джерелом), де ймовірнісний розподіл пар літер залишається стабільним у всьому масиві. Завдяки цій стабільності зменшення кількості аналізованих пар удвічі при використанні неперекривних біграм не призводить до втрати загальних закономірностей мови, дозволяючи обох методам давати однаково достовірну оцінку.

3. Дослідження впливу розподілу символів на ентропію

Підготуйте три послідовності достатньої для проведення експерименту довжини:

- А — фрагмент природного російськомовного тексту;
- Б — послідовність, утворену повторенням одного символу;
- В — випадкову послідовність символів того самого алфавіту, в якій усі символи генеруються з однаковою ймовірністю.

До виконання обчислень розташуйте послідовності А, Б і В у порядку очікуваного зростання H_1 та коротко обґрунтуйте свій прогноз.

За допомогою власної програми обчисліть H_1 для кожної послідовності.

Порівняйте отримані результати з попереднім прогнозом. Поясніть, як характер розподілу ймовірностей символів впливає на ентропію джерела.

Порядок очікуваного зростання H_1 :

$B < A < V$

Б (повторення одного символу) — ансамбль вироджений: один символ має ймовірність $p=1$, всі інші $p=0$. За визначенням, $H(Z)=0$ саме в цьому випадку $\rightarrow H_1(B)=0$, мінімум із можливих значень.

А (природний текст) — мови мають нерівноймовірний розподіл літер (наприклад, «о», «е», «а» трапляються значно частіше за «ф», «щ», «ъ»). Тому $H_1(A)$ буде проміжним значенням: більшим за 0 (бо є розподіл, а не виродженість), але меншим за H_0 (бо розподіл нерівномірний).

В (рівноймовірна випадкова послідовність) — усі символи алфавіту мають однакову ймовірність $p=1/n$. Це саме той випадок, коли ентропія набуває максимального значення.

Програма:

```
task3.py > compute_entropy
1  import math
2  import re
3  import random
4  from collections import Counter
5
6  CUSTOM_ALPHABET = "абвгдежзийклмнопрстуфхцчщъыюя"
7  ALLOWED_CHARS = set(CUSTOM_ALPHABET + " ")
8  ALPHABET_LIST = list(ALLOWED_CHARS)
9
10 def clean_text(raw_data: str, keep_spaces: bool = True) -> str:
11     text = raw_data.lower()
12     replacements = {"ё": "е", "ь": "ъ", "э": "е"}
13     for old_char, new_char in replacements.items():
14         text = text.replace(old_char, new_char)
15     filtered_chars = [ch if ch in ALLOWED_CHARS else " " for ch in text]
16     text = "".join(filtered_chars)
17     text = re.sub(r"\s+", " ", text).strip()
18     if not keep_spaces:
19         text = text.replace(" ", "")
20     return text
21
```

функція бере сирий текст, приводить до нижнього регістру, замінює ё/ь/э на їхні еквіваленти, викидає все, що не входить в алфавіт (заміняючи на пробіл), стискає послідовності пробілів в один

```
1
2  def compute_entropy(counts: Counter, total: int) -> float:
3      entropy = 0.0
4      for count in counts.values():
5          p = count / total
6          entropy -= p * math.log2(p)
7      return entropy
8
9  def compute_entropy_h1(text: str) -> float:
10     counts = Counter(text)
11     return compute_entropy(counts, len(text))
12
```

Функція `compute_entropy_h1` підраховує кількість входжень кожного символу в тексті (за допомогою `Counter`) та передає ці частоти у функцію `compute_entropy` разом із загальною довжиною тексту.

Функція `compute_entropy` реалізує класичну формулу ентропії Шеннона:

Для кожного унікального символу обчислюється його ймовірність p (кількість входжень, поділена на загальну довжину тексту), після чого відповідний доданок

$p(\log_2 p)$ додається до суми (з врахуванням знака "мінус" перед сумою за формулою). Результатом є значення ентропії H_1 у бітах на символ.

```
def task3(natural_text: str, n: int, seed: int = 42):
    random.seed(seed)

    # A - фрагмент природного тексту (випадковий зсув у межах файлу)
    start = random.randint(0, max(0, len(natural_text) - n - 1))
    seq_A = natural_text[start:start + n]

    # Б - повторення одного символу
    most_common_char = Counter(natural_text).most_common(1)[0][0]
    seq_B = most_common_char * n

    # В - випадкова рівномірна послідовність
    seq_V = "".join(random.choices(ALPHABET_LIST, k=n))

    print(f"{'Послідовність':35s} {'H1 (біт/символ)':>18s}")
    print("-" * 55)
    for name, seq in [
        ("А (природний текст)", seq_A),
        ("Б (один символ, повтор)", seq_B),
        ("В (рівномірний випадковий)", seq_V)]:
        h1 = compute_entropy_h1(seq)
        print(f"{name:35s} {h1:18.5f}")

    max_h0 = math.log2(len(ALPHABET_LIST))
    print(f"Максимум ентропії: |алфавіт| = {len(ALPHABET_LIST)}, H0 (максимум) = {max_h0:.5f} біт/символ")
```

Функція task3 формує три тестові послідовності на одному й тому ж алфавіті: А — випадковий фрагмент довжини n із реального тексту (через випадковий зсув start, щоб уникнути упередженості вибірки); Б — послідовність довжини n , що повністю складається з найчастішої літери природного тексту (most_common(1)); В — послідовність довжини n , згенерована випадковим вибором символів алфавіту з рівною ймовірністю кожного (random.choices). Для кожної обчислюється H_1 за раніше реалізованою функцією, а також виводиться теоретичний максимум для порівняння.

```
if __name__ == "__main__":
    filename = "Pyat_lojek_eliksira.txt"
    N = 100000

    with open(filename, encoding="utf-8", errors="ignore") as f:
        raw = f.read()

    natural = clean_text(raw, keep_spaces=True)
    task3(natural, N)
```

Головний блок програми відкриває текстовий файл Pyat_lojek_eliksira.txt у кодуванні UTF-8, зчитує його вміст, після чого нормалізує текст функцією clean_text (з урахуванням пробілів) і передає результат у функцію task3 разом із заданою довжиною послідовностей $N=100000$ для проведення експерименту.

Результат:

Послідовність	H_1 (біт/символ)
А (природний текст)	4.35881
Б (один символ, повтор)	0.00000
В (рівномірний випадковий)	4.95403
Довідково: $ \text{алфавіт} = 31$, H_0 (максимум) = 4.95420 біт/символ	

Отримані значення H_1 повністю підтверджують теоретичний прогноз $B < A < V$ ($0.00000 < 4.35881 < 4.95403$). Послідовність Б, що складається з одного символу, має нульову ентропію через відсутність будь-якої невизначеності. Послідовність В, з рівномірним розподілом символів, демонструє ентропію, практично рівну теоретичному максимуму $H_0 = \log_2(31) = 4.95420$. Природний текст А займає проміжне положення ($H_1 = 4.35881$) через нерівномірність розподілу літер у мові. Це підтверджує, що характер розподілу ймовірностей символів на пряму визначає ентропію джерела: чим рівномірніший розподіл — тим вища ентропія (максимум при рівномірності), а чим сильніше він зміщений у бік одного чи кількох доміантних символів — тим нижча ентропія (мінімум при виродженому розподілі). Природна мова займає проміжне положення саме через цю нерівномірність, що й лежить в основі її надлишковості.

4. Дослідження залежностей між символами

Сформуйте дві послідовності однакової довжини, побудовані на однаковому алфавіті

та з однаковими або максимально близькими частотами окремих символів:

- Г — символи розташовані у випадковому порядку;
- Д — символи утворюють виражену періодичну структуру, наприклад abababab....

До виконання обчислень сформулюйте прогноз:

1. чи повинні суттєво відрізнятися значення H_1 для послідовностей Г і Д;
2. чи повинні суттєво відрізнятися значення H_2 ;
3. чи для якої послідовності ви очікуєте менше значення H_2 .
4. чи відрізнятимуться H_2 обчислене для біграм що перетинаються і що не перетинаються

Обчисліть H_1 та H_2 для обох послідовностей.

Порівняйте результати з прогнозом та поясніть:

- чому тексти з однаковими частотами окремих символів можуть мати однакові або близькі значення H_1 ;
- чому їхні значення H_2 можуть відрізнятися;
- яку інформацію про джерело дозволяє врахувати модель біграм порівняно з моделлю незалежних символів.

1. H_1 для Γ і D не повинні суттєво відрізнятися. H_1 залежить лише від частот окремих символів (скільки разів «а» і скільки разів «б» зустрілося), а не від їхнього порядку. Оскільки частоти в Γ і D однакові за умовою завдання — $H_1(\Gamma) \approx H_1(D)$.

2. H_2 повинні суттєво відрізнятися. H_2 враховує зв'язки між сусідніми символами (біграми), а порядок символів у Γ і D принципово різний: у Γ — випадковий, у D — жорстко детермінований (завжди «аб» або «ба» по черзі).

3. Менше значення H_2 очікується для D . У періодичній структурі наступний символ повністю передбачуваний за попереднім (після «а» завжди йде «б») — це майже нульова невизначеність. У Γ порядок випадковий, тому передбачити наступний символ важче — вища невизначеність.

4. Для D очікується суттєва відмінність між H_2 , обчисленою для перекривних біграм, і H_2 для неперекривних біграм. Якщо біграми брати без перекриття (зміщуючись щоразу на два символи), то з послідовності «абабаб...» завжди вибиратимуться біграми лише одного типу («аб» щоразу) — розподіл вироджений, $H_2 \rightarrow 0$. Якщо ж біграми брати з перекриттям (зміщуючись щоразу лише на один символ), трапляються по чергові і «аб», і «ба» — розподіл менш вироджений, тому H_2 вища. Для Γ такої різниці не очікується, бо там немає прихованої періодичності.

Програма:

```

task4.py > task4
1  import math
2  import random
3  from collections import Counter
4
5  def compute_entropy(counts: Counter, total: int) -> float:
6      entropy = 0.0
7      for count in counts.values():
8          p = count / total
9          entropy -= p * math.log2(p)
10     return entropy
11
12 def compute_entropy_h1(text: str) -> float:
13     counts = Counter(text)
14     return compute_entropy(counts, len(text))
15
16 def get_overlapping_bigrams(text: str) -> list:
17     return [text[i:i+2] for i in range(len(text) - 1)]
18
19 def get_nonoverlapping_bigrams(text: str) -> list:
20     return [text[i:i+2] for i in range(0, len(text) - 1, 2)]
21
22 def calculate_h2_per_symbol(bigrams_list: list) -> float:
23     counts = Counter(bigrams_list)
24     total = sum(counts.values())
25     h_joint = compute_entropy(counts, total)
26     return h_joint / 2.0
27

```

Функції `get_overlapping_bigrams` і `get_nonoverlapping_bigrams` формують список біграм (пар сусідніх символів) із тексту двома способами: з кроком зсуву 1 (пари перекриваються спільним символом) та з кроком зсуву 2 (пари йдуть послідовно без перекриття). Функція `calculate_h2_per_symbol` підраховує частоти унікальних біграм, обчислює їхню сукупну ентропію за формулою Шеннона (як для окремого алфавіту пар символів), після чого ділить результат на 2, отримуючи питому ентропію H_2 у розрахунку на один символ тексту.

```

29 def task4(n: int, symbols=("a", "b"), seed: int = 42):
30     random.seed(seed)
31     s1, s2 = symbols
32     half = n // 2
33
34     # Г — випадковий порядок
35     pool = [s1] * half + [s2] * (n - half)
36     random.shuffle(pool)
37     seq_G = "".join(pool)
38
39     # Д — виражена періодична структура
40     seq_D = (s1 + s2) * (n // 2)
41     if len(seq_D) < n:
42         seq_D += s1
43
44     print(f"'Послідовність':30s {'H1':>10s} {'H2 (перекр.):>14s} {'H2 (неперекр.):>16s}")
45     print("-" * 74)
46     for name, seq in [("Г (випадковий порядок)", seq_G), ("Д (періодична abab...)", seq_D)]:
47         h1 = compute_entropy_h1(seq)
48         h2_over = calculate_h2_per_symbol(get_overlapping_bigrams(seq))
49         h2_nonover = calculate_h2_per_symbol(get_nonoverlapping_bigrams(seq))
50         print(f"{name:30s} {'h1:10.5f'} {'h2_over:14.5f'} {'h2_nonover:16.5f}")
51
52     print(f"\nЧастоти символів Г: {dict(Counter(seq_G))}")
53     print(f"Частоти символів Д: {dict(Counter(seq_D))}")
54
55
56 if __name__ == "__main__":
57     N = 100000
58     task4(N)

```

Функція task4 формує дві послідовності однакової довжини n з однаковими частотами символів «а» і «б» (по n/2 кожного): Г — символи перемішуються у випадковому порядку (random.shuffle), Д — символи чергуються строго періодично («абабаб...»). Для обох послідовностей обчислюються H1, а також H2 двома способами (для перекривних і неперекривних біграм), результати виводяться у вигляді таблиці, а додатково друкуються частоти символів для перевірки коректності умови завдання.

Результат:

Послідовність	H1	H2 (перекр.)	H2 (неперекр.)
Г (випадковий порядок)	1.00000	1.00000	0.99999
Д (періодична abab...)	1.00000	0.50000	0.00000
Частоти символів Г: {'a': 50000, 'b': 50000}			
Частоти символів Д: {'a': 50000, 'b': 50000}			

Порівняння з прогнозом

Усі пункти прогнозу підтвердились: H1 для Г і Д практично однакова (1.00000), H2 суттєво відрізняється, менше значення H2 — у послідовності Д, і саме для Д спостерігається суттєва різниця між перекривними й неперекривними біграмами (0.5 проти 0.0), тоді як для Г ця різниця мізерна (1.00000 проти 0.99999).

Чому тексти з однаковими частотами символів можуть мати однакові або близькі H_1

H_1 обчислюється виключно на основі ймовірностей окремих символів (скільки разів кожен символ зустрівся), і формула жодним чином не враховує, у якому порядку ці символи розташовані в тексті. Оскільки в Г і Д частоти символів «а» і «б» абсолютно однакові (по 50000 кожного), ймовірності $p("a")=p("b")=0.5$ в обох випадках — тому й H_1 обов'язково збігається, незалежно від того, наскільки різним є внутрішній порядок символів.

Чому їхні значення H_2 можуть відрізнитися

H_2 обчислюється на основі частот пар сусідніх символів (біграм), а не лише окремих символів, і тому вловлює залежність між сусідніми елементами послідовності. У Д ця залежність максимальна — наступний символ повністю визначається попереднім (після «а» завжди йде «б»), тому реальна кількість "унікальних" варіантів пар мінімальна, і H_2 низька. У Г символи розташовані випадково, наступний символ не залежить від попереднього, тому пари близькі до незалежних, і H_2 залишається високою. Однакові частоти окремих символів не гарантують однакової структури пар — саме тому H_2 може відрізнитись навіть при рівних H_1 .

Яку інформацію додає модель біграм порівняно з моделлю незалежних символів

Модель незалежних символів (H_1) враховує лише те, які символи присутні в тексті та з якою частотою, повністю ігноруючи їхнє взаємне розташування. Модель біграм (H_2) додатково враховує статистичний зв'язок між сусідніми символами — наскільки передбачуваним є наступний символ, знаючи попередній. Це дозволяє виявляти приховану структуру джерела (закономірності, повтори, залежності), яку модель незалежних символів у принципі не здатна помітити, і завдяки цьому отримати точнішу (як правило, нижчу) оцінку справжньої ентропії джерела — саме цей принцип переходу від простіших до складніших моделей і лежить в основі точнішого наближення ентропії мови.

5. Експериментальна оцінка ентропії джерела

За допомогою програми CoolPinkProgram оцініть значення

$H^{(10)}, H^{(20)}, H^{(30)}$.

Для оцінювання кожного значення виконайте не менше 50 експериментів. Порівняйте отримані значення з H_1 та H_2 , обчисленими власною програмою. Почну з $H^{(10)}$

Лабораторная работа №1

Произвольная часть текста:
но_в_дан

Использованные буквы:

Порядок n-граммы:
5 символов
10 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 51

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:
 $3,62871847518549 < H < 4,10768348363125$

Двоичная таблица угаданных символов:

10000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
10000000000000000000000000000000
01000000000000000000000000000000
00000000000000000000000000000000

Вероятности:

$q[1] = 0,22$
$q[2] = 0,08$
$q[3] = 0,04$
$q[4] = 0,08$
$q[5] = 0,04$
$q[6] = 0,02$
$q[7] = 0$
$q[8] = 0,02$
$q[9] = 0,04$
$q[10] = 0,04$
$q[11] = 0,02$
$q[12] = 0,06$
$q[13] = 0$
$q[14] = 0$
$q[15] = 0,04$
$q[16] = 0,02$
$q[17] = 0$
$q[18] = 0,02$
$q[19] = 0$
$q[20] = 0,02$
$q[21] = 0,02$
$q[22] = 0$
$q[23] = 0$
$q[24] = 0,02$
$q[25] = 0,02$
$q[26] = 0,02$
$q[27] = 0,02$
$q[28] = 0,08$
$q[29] = 0,02$
$q[30] = 0,02$
$q[31] = 0,02$
$q[32] = 0$

Строка состояния:

Для оцінки $H^{(10)}$ проведено 50 експериментів. Експериментальна оцінка за допомогою програми CoolPinkProgram становить $3,6287 < H^{(10)} < 4,1077$ біт/символ (середнє значення $\approx 3,868$).

Продовжуємо для $H^{(20)}$

Лабораторная работа №1

Произвольная часть текста:
а_и_нигде_не_читал

Использованные буквы:

Порядок n-граммы:
5 символов
10 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 51

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:
 $2,56717654734645 < H < 3,44689006271891$

Двоичная таблица угаданных символов:

10000000000000000000000000000000
00001000000000000000000000000000
00010000000000000000000000000000
00000010000000000000000000000000
10000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000

Вероятности:

$q[1] = 0,42$
$q[2] = 0,1$
$q[3] = 0,02$
$q[4] = 0,04$
$q[5] = 0,02$
$q[6] = 0,04$
$q[7] = 0,02$
$q[8] = 0,02$
$q[9] = 0$
$q[10] = 0,02$
$q[11] = 0,02$
$q[12] = 0,02$
$q[13] = 0,02$
$q[14] = 0,02$
$q[15] = 0,02$
$q[16] = 0$
$q[17] = 0,02$
$q[18] = 0$
$q[19] = 0,02$
$q[20] = 0$
$q[21] = 0,02$
$q[22] = 0,02$
$q[23] = 0,02$
$q[24] = 0,02$
$q[25] = 0,04$
$q[26] = 0,02$
$q[27] = 0,02$
$q[28] = 0$
$q[29] = 0$
$q[30] = 0$
$q[31] = 0$
$q[32] = 0$

Строка состояния:

Експериментальна оцінка становить $2,5672 < H^{(20)} < 3,4469$ біт/символ (середнє значення $\approx 3,007$).

І тепер для $H^{(30)}$

Лабораторная работа №1

Произвольная часть текста:
ч_тяготения_и_не_может_пойти_

Использованные буквы:

Порядок n-граммы:
5 символов
10 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ:
Символ по счету:
Номер эксперимента: 52

Поле ввода символов:

Продолжить

Другой

Неравенство для энтропии:
1,40437367983374 < H < 2,06311854955452

Двоичная таблица угаданных символов:
01000000000000000000000000000000
10000000000000000000000000000000
00000000001000000000000000000000
01000000000000000000000000000000
00100000000000000000000000000000

Вероятности:
q[1] = 0,6470588
q[2] = 0,0980392
q[3] = 0,0196078
q[4] = 0,0392156
q[5] = 0
q[6] = 0,0196078
q[7] = 0
q[8] = 0
q[9] = 0
q[10] = 0,019607
q[11] = 0,039215
q[12] = 0,019607
q[13] = 0
q[14] = 0
q[15] = 0,019607
q[16] = 0,019607
q[17] = 0
q[18] = 0
q[19] = 0
q[20] = 0
q[21] = 0
q[22] = 0
q[23] = 0
q[24] = 0
q[25] = 0
q[26] = 0,019607
q[27] = 0,039215
q[28] = 0
q[29] = 0
q[30] = 0
q[31] = 0
q[32] = 0

Строка состояния:

Для n = 30 вийшло: $1,40437 < H^{(30)} < 2,06312$, середина $\approx 1,734$ біт/символ.

Експериментів було 51

Результати подайте в одній порівняльній таблиці:

Оцінка	Отримане значення
H_1	4,3583
H_2	3,9891
$H^{(10)}$	середнє $\approx 3,868$

$H^{(20)}$	середнє $\approx 3,007$
$H^{(30)}$	середнє $\approx 1,734$

H_1 і H_2 взяті з завдання 2 для тексту з пробілами

Аналіз результатів

1. Чому оцінки не повинні збігатися. Це різні величини. H_1 і H_2 обчислені точно за частотами літер і пар у великому тексті (майже 700 тисяч символів). $H^{(n)}$ це умовна ентропія при контексті довжиною $n-1$ символів, і її оцінено експериментально, за 50 спробами, для іншого тексту. Вибірка невелика, тож результат ще й трохи коливається від запуску до запуску.

2. Які залежності враховують H_1 та H_2 . H_1 враховує лише нерівномірність окремих літер (о, е, а й пробіл трапляються частіше за ф чи щ), а зв'язки між літерами не враховує зовсім. H_2 додатково враховує залежність між двома сусідніми літерами, тобто вплив попередньої літери на наступну. Зв'язків довших за одну літеру вона не бачить.

3. Чому довші залежності змінюють оцінку. Природна мова має структуру на всіх рівнях: літери складаються в склади й слова, слова підпорядковуються граматиці та змісту. Що довший контекст, то менше варіантів залишається для наступної літери: за 29 попередніми символами зазвичай видно слово, а часто й усю фразу, і продовження майже очевидне. Умовна ентропія не зростає, коли додається більше умов, тому врахування довших залежностей зменшує оцінку і наближає її до справжньої ентропії мови.

4. Тенденція. Оцінка монотонно спадає зі збільшенням контексту: $H_1 (4,36) > H_2 (3,99) > H^{(10)} (\approx 3,87) > H^{(20)} (\approx 3,01) > H^{(30)} (\approx 1,73)$. Найбільше падіння відбувається саме при переході до довгого контексту (20-30 символів), коли стають передбачуваними цілі слова та звороти. У межі така послідовність прямує до ентропії мови. Надлишковість природної мови велика, адже ентропія на символ значно менша за $H_0 = \log_2 31 \approx 4,954$.

6. Оцінювання надлишковості джерела

Використовуючи отримані значення ентропії, оцініть надлишковість російського

письмового тексту для досліджених моделей джерела.

Порівняйте отримані оцінки та поясніть:

- як змінюється оцінка надлишковості при врахуванні залежностей між символами;
- чому природна мова має ненульову надлишковість;
- який зв'язок існує між передбачуваністю наступних символів тексту, ентропією та надлишковістю джерела.

Надлишковість джерела: $R = 1 - H / H_0$, де $H_0 = \log_2 31 \approx 4,9542$ біт/символ (алфавіт із 30 літер і пробілу, усі символи рівноймовірні та незалежні).

Таблиця

Модел ь	H (біт/символ)	R
H_1	4,3583	0,120 (12,0%)
H_2	3,9891	0,195 (19,5%)
$H^{(10)}$	$\approx 3,868$	$\approx 0,219$
$H^{(20)}$	$\approx 3,007$	$\approx 0,393$
$H^{(30)}$	$\approx 1,734$	$\approx 0,650$

Як змінюється оцінка надлишковості при врахуванні залежностей між символами. Вона зростає: від 12% для H_1 до 19,5% для H_2 , далі до $\approx 22\%$, $\approx 39\%$ і $\approx 65\%$ для $H^{(10)}$, $H^{(20)}$, $H^{(30)}$. Причина проста: чим більше залежностей ми враховуємо, тим менше значення ентропії та тим більша різниця між ним і максимумом H_0 . Модель H_1 бачить лише нерівноймовірність літер, тому «пропускає» більшу частину надлишковості, а довший контекст виявляє її дедалі більше. Для $H^{(30)}$ значення $\approx 65\%$ вже близьке до класичних оцінок надлишковості природних мов (близько 60–75%), але воно, найімовірніше, лише нижня межа справжньої надлишковості, бо ентропія в експерименті завищена: людина вгадує гірше за ідеальний передбачувач. Справжня надлишковість ще вища.

Чому природна мова має ненульову надлишковість. Тому що символи в ній не рівноймовірні та не незалежні. По-перше, літери мають різну частоту (о, е, а, пробіл трапляються значно частіше за ф чи щ). По-друге, між літерами є зв'язки:

закони складів і слів, граматики, зміст (після «чт» майже напевно йде «о», а «ъ» не може стояти на початку слова). Через це не всі комбінації символів можливі, і реальна ентропія значно менша за H_0 . Надлишковість корисна: вона дозволяє відновити слово з пошкодженими літерами й виправляти помилки та описки, але і є ресурсом, який можна «вичавити» стисненням.

Зв'язок між передбачуваністю, ентропією та надлишковістю. Чим передбачуваніший наступний символ, тим менша невизначеність джерела, тобто тим менша ентропія, і тим більша надлишковість. У нашому експерименті це видно прямо: $q[1]$, частка літер, вгаданих з першої спроби, зростає від 0,22 ($n=10$) до 0,42 ($n=20$) і 0,65 ($n=30$), і разом з цим падає ентропія та росте R . Надлишковість, по суті, показує, яка частка тексту «не несе нової інформації», бо її можна вгадати з контексту, а отже, її можна стиснути без втрати змісту.

Аналіз результатів

1. Чому природний текст має меншу ентропію на символ, ніж випадкова рівноймовірна послідовність символів того самого алфавіту?

Ентропія максимальна, коли всі символи рівноймовірні. У послідовності В так і є: $H_1 = 4,954 \approx H_0 = \log_2 31$. У природному тексті А літери зустрічаються з різною частотою: пробіл має ймовірність 0,161, «о» 0,091, «е» 0,076, тоді як рівномірна ймовірність була б $1/31 \approx 0,032$. Частина символів стала більш передбачуваною, тому невизначеність менша: $H_1(A) = 4,359$, тобто на 0,6 біта нижче за В.

2. Яку інформацію про джерело враховує H_2 , але не враховує H_1 ?

H_1 спирається лише на частоти окремих літер, а H_2 ще й на залежність між двома сусідніми символами: яка літера вірогідно йде після якої. У тексті це видно за найчастішими біграмами («о_», «е_», «_п», «_н», «_в»), які пов'язані з початками й кінцями слів. Через цю залежність $H_2 = 3,989$ менше за $H_1 = 4,358$ приблизно на 0,37 біта.

3. Чи можуть два тексти мати однакове H_1 , але різне H_2 ? Наведіть приклад із виконаних експериментів.

Так, це показує завдання 4. Послідовності Г (випадковий порядок) і Д (періодична «абаб...») мають однакові частоти (по 50000 «а» і «б»), тому $H_1 = 1,00000$ в обох. Але H_2 різне: у Г вона 1,00000, а в Д 0,50000 (біграми, що перетинаються) і 0,00000 (що не перетинаються). H_1 не бачить порядку символів, а H_2 бачить.

4. Як вилучення пробілів вплинуло на отримані значення ентропії? Поясніть отриманий результат.

Ентропія зростає: H_1 з 4,358 до 4,435, H_2 з 3,989 до 4,148.

Пробіл найчастіший символ (16%), тому він робить розподіл нерівномірним. Без нього літери розподілені рівніше, і H_1 росте. Крім того, пробіл робить біграми передбачуванішими (закінчення й початки слів, «о_», «_п»). Без пробілів пари через межі слів стають менш очевидними, і H_2 зростає навіть сильніше.

Надлишковість, відповідно, зменшується: за H_2 з 19,5% до $\approx 15,5\%$.

5. Чому оцінки H_2 , отримані за біграмами, що перетинаються та не перетинаються, можуть дещо відрізнятися?

Для тексту з пробілами: 3,98909 проти 3,98853 (різниця 0,00056), а без пробілів збіглося (4,14824). Способи використовують різну вибірку: 690625 біграм проти 345313, і неперетинні біграми беруть лише пари з парних позицій. Емпірична оцінка ентропії трохи залежить від обсягу вибірки й від того, які саме пари потрапили в підрахунок, тому виникає дрібна статистична розбіжність. Для великого й однорідного тексту вона мізерна, і перший спосіб надійніший, бо використовує більше даних. Проте для періодичної послідовності D різниця величезна (0,5 проти 0), бо при кроці 2 підрахунок «застряє» в одній фазі періоду.

6. Чому значення H_1 , H_2 , $H^{(10)}$, $H^{(20)}$ та $H^{(30)}$ відрізняються?

Вони враховують різну довжину контексту: H_1 нуль попередніх символів, H_2 один, $H^{(n)}$ $n-1$ символів. Чим довший контекст, тим менше варіантів залишається для наступної літери, і умовна ентропія падає: $4,36 > 3,99 > \approx 3,87 > \approx 3,01 > \approx 1,73$. Крім того, H_1 і H_2 пораховані точно за 690 тис. символів, а $H^{(n)}$ оцінені експериментально.

7. Який із проведених експериментів найкраще демонструє наявність залежностей між послідовними символами?

Найяскравіше це завдання 4: за однакового $H_1 = 1$ значення H_2 падає з 1,0 (Г) до 0,5 і навіть 0 (Д), тобто залежність між сусідніми символами видно лише в H_2 . На реальному тексті це підтверджує завдання 5: частка літер, вгаданих з першої спроби, зростає з 22% при $n = 10$ до 42% при $n = 20$ і 65% при $n = 30$. Що довший контекст, то краще вгадується наступний символ.

8. Як отримані результати пов'язані з можливістю стиснення природного тексту без втрати інформації?

Ентропія задає теоретичну межу: середня довжина коду не може бути меншою за H біт на символ. Без стиснення на символ треба $\log_2 31 \approx 4,95$ біта, а наша оцінка з урахуванням довгого контексту, $H^{(30)} \approx 1,7$ біта, дає надлишковість $\approx 65\%$, тобто в принципі текст можна стиснути в кілька разів. Модель H_1 дозволяє лише $\approx 12\%$

(4,36 проти 4,95 біта), $H_2 \approx 19,5\%$. Це узгоджується з тим, що архіватори, які враховують довгі залежності між символами, стискають текст значно краще, ніж кодування за частотами літер. Випадкова послідовність В, навпаки, не стискається, бо її H_1 дорівнює H_0 і надлишковості немає.

Висновок по роботі:

У ході роботи експериментально оцінено ентропію на символ російського письмового тексту та її залежність від контексту й внутрішньої структури мови. Зі збільшенням глибини врахування міжсимвольних зв'язків оцінка невизначеності монотонно спадає, а частка вгаданих з першої спроби літер зростає з 22% до 65%, що відображає наближення до граничної ентропії.

Найчастіший символ — пробіл — суттєво знижує невизначеність і робить послідовність передбачуванішою, тому його вилучення підвищує ентропію ($H_1 = 4,435$, $H_2 = 4,148$). Завдяки великому обсягу та однорідності вибірки (1,27 МБ, 690 тис. символів) оцінки H_2 для біграм, що перетинаються та не перетинаються, виявилися майже ідентичними (3,98909 проти 3,98853).

Експерименти підтвердили, що модель H_1 залежить виключно від частот символів і не бачить порядку їх прямокутного, тоді як модель біграм (H_2) та метод вгадування успішно фіксують контекстні залежності. У результаті обчислена надлишковість мови зростає від 12% (H_1) і 19,5% (H_2) до 39% ($H^{(20)}$) та 65% ($H^{(30)}$).

Ця надлишковість зумовлена нерівномірністю літер і граматичною структурою мови, що створює теоретичне підґрунтя для ефективного стиснення тексту без втрати інформації.